Northeastern University, Khoury College of Computer Science

CS 6120 Natural Language Processing — Lab T10.1

March 16, 2026 (Week 10)

LLM Serving and User Interface on HPC

In this lab, we're going to serve our own Large Language Model with Ollama using the Streamlit front-end library. Large Language Models work best with GPUs because Transformers are computationally expensive. If you do not have one, you can use Northeastern's Explorer HPC cluster's GPU resources. Most of the material that you'll need can be found in this repository, a containerized solution.

Key differences from the GCP version:

- We use Apptainer instead of Docker
- We use SSH tunneling to access the web interface (HPC nodes don't have public IPs)
- We build from an Apptainer definition file instead of a Dockerfile

1 Setup: Access HPC GPU Node

1. SSH to HPC:

```
ssh your.username@login.explorer.northeastern.edu
```

2. Request a GPU node:

```
srun -p courses-gpu --gres=gpu:1 --time=4:00:00 \
     --mem=16GB --cpus-per-task=4 --pty /bin/bash
```

This allocates 1 GPU, 16GB RAM, 4 CPU cores, and 4 hours runtime. Note the node name that Slurm assigns you (e.g., `c2193`) — you will need this for SSH tunneling later. You can confirm which GPU you received by running `nvidia-smi` once on the node.

3. Set up scratch storage for Ollama models:

   The home directory (`/home`) has a strict 75GB quota which is not enough for LLM weights. We use `/scratch` instead, which gives you up to 20TB of temporary storage.

```
mkdir -p /scratch/$USER/ollama_models
```

   Note: *`/scratch` is purged on the first Tuesday of every month. If your models disappear between sessions, simply re-run the container; they will re-download automatically. Do not store anything important here permanently.*

4. Clone the provided repository:

```
cd /courses/CS6120.202630/students/your.username
git clone https://github.com/kni-neu/serving-llama
cd serving-llama
```

2 Build the Container with Apptainer

HPC systems use Apptainer (formerly Singularity) instead of Docker for security reasons.

1. Create Apptainer definition file:

   The definition file is the Apptainer equivalent of the Dockerfile in the GCP version; it defines how to build your container. Apptainer can bootstrap directly from Docker Hub images, so we use the official Ollama Docker image as the base for the definition file (the same `ollama/ollama` image the Dockerfile uses for the GCP version).

```
cat > serving-llama.def << 'EOF'
Bootstrap: docker
From: ollama/ollama

%files
    requirements.txt /root/requirements.txt
    app.py /root/app.py
    entrypoint.sh /root/entrypoint.sh

%post
    apt update
    apt-get install -y python3 python3-pip vim git
    pip install --break-system-packages -r /root/requirements.txt
    chmod +x /root/entrypoint.sh

%runscript
    cd /root
    exec ./entrypoint.sh
EOF
```

   This definition file:

- Bootstraps from the official Ollama Docker image. "Bootstrap" here just means starting point. In practice, almost everyone starts from a pre-built base image rather than from scratch, because it saves a lot of work. Docker Hub is an online registry, a website where pre-built container images are stored and shared publicly. Think of it like GitHub, but for container images instead of code. Both Docker and Apptainer can pull from it
- Copies app.py, requirements.txt, and entrypoint.sh from the cloned repository into the container
- Installs Python and dependencies
- Sets the entrypoint script to run on container startup

2. Set up cache directory:

   When Apptainer builds a container, it needs to store temporary files and cache downloaded layers somewhere. By default it tries to use the home directory (~), but on Northeastern's HPC cluster, home directories have a strict storage quota, enough for code and small files, not for large container images.

```
mkdir -p /scratch/$USER/apptainer_cache
export APPTAINER_CACHEDIR=/scratch/$USER/apptainer_cache
export APPTAINER_TMPDIR=/scratch/$USER/apptainer_cache
```

   `/scratch` has much more space than `/home`, so these two environment variables redirect Apptainer's cache there instead:

   - `APPTAINER_CACHEDIR`: where Apptainer stores downloaded base images (like `ollama/ollama`)
   - `APPTAINER_TMPDIR`: where Apptainer stores temporary files during the build process

3. Build the container:

```
apptainer build serving-llama.sif serving-llama.def
```

   This will take several minutes as it reads `serving-llama.def` that we created, pulls the Ollama image from Docker Hub, installs packages, and creates the `.sif` container.

3 Proxy Configuration

Explorer HPC runs a Squid proxy that manages all outbound HTTP/HTTPS traffic. This creates two requirements that must both be satisfied:

- The proxy must be passed into the container so that `ollama pull` can reach `ollama.com` to download models
- The proxy must be bypassed for localhost so it does not intercept Ollama's internal communication between the server and Streamlit

Both are handled by the `--env` flags in the `apptainer run` command in Section 4, Step 3.

4 Serve Your LLM

You will notice that in the Definition file, the `%runscript` section calls `entrypoint.sh`, a script that runs when the container starts. It downloads the model you wish to serve and deploys the Streamlit application. The script `entrypoint.sh` defaults to pulling Llama 3, which is a bit dated. Choose another model from the list of available LLMs.

The Streamlit application is a front-end that plays nicely with Python. Feel free to add a few fields (maybe a nice picture) in `app.py`. This Python file gets called every time you enter text into the prompt on the webpage. You can perhaps guess what `st.sidebar` or `st.caption` does. For instance, the default `app.py` says "powered by OpenAI"; since you are running Ollama locally, you may want to update this. For more documentation, visit Streamlit's website.

1. Choose a model and edit `entrypoint.sh`:

   To use a different model, edit `entrypoint.sh` and change:

   ```
   ollama pull llama3
   ```

   Browse available models at https://ollama.com/search.

2. Rebuild the container:

   After modifying any files, rebuild the container before running:

   ```
   apptainer build serving-llama.sif serving-llama.def
   ```

3. Run the container with GPU support:

   Deploy your LLM to Streamlit using this command:

   ```
   apptainer run --nv \
     --bind "/scratch/$USER:/scratch/$USER" \
     --env OLLAMA_MODELS=/scratch/$USER/ollama_models \
     --env HTTP_PROXY=$HTTP_PROXY \
     --env HTTPS_PROXY=$HTTPS_PROXY \
     --env NO_PROXY=localhost,127.0.0.1,0.0.0.0 \
     serving-llama.sif
   ```

   The `--nv` flag enables NVIDIA GPU support inside the container.

   You should see:

   - Ollama server starting on port 11434
   - GPU detected (your assigned GPU)
   - Streamlit server starting on port 8501

   Check to see if it's running on GPU. In another SSH session to the GPU node:

   ```
   nvidia-smi
   ```

4. Set up SSH tunnel:

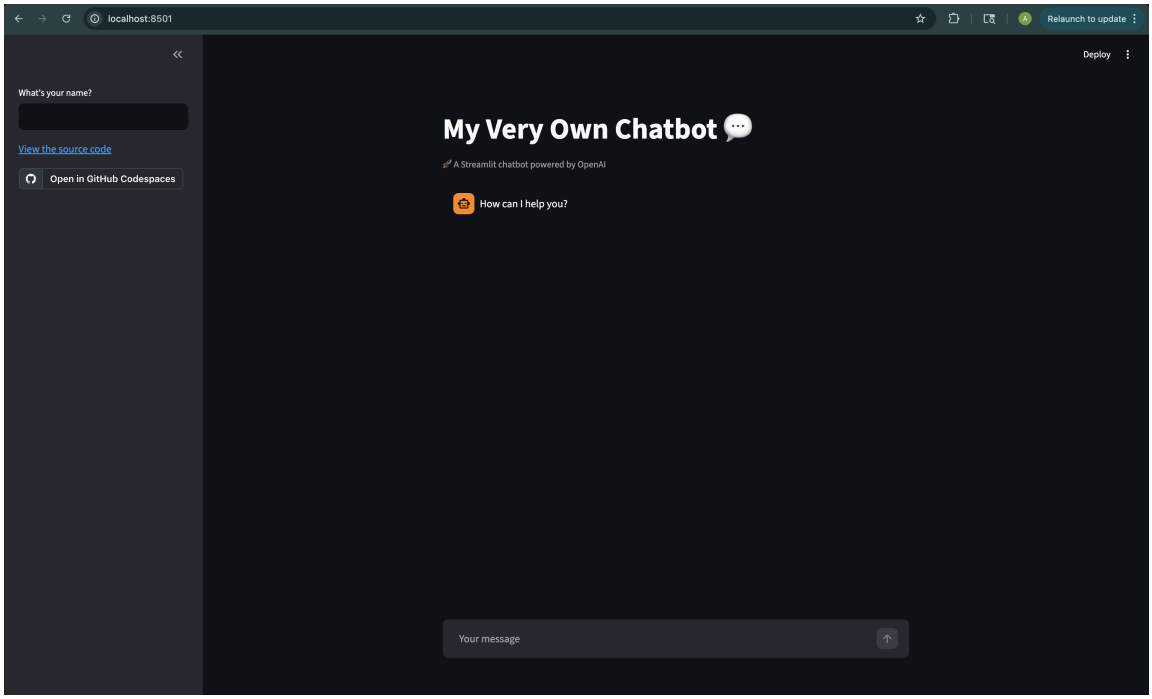
   In a new terminal on your local machine:

```
ssh -L 8501:c2193:8501 your.username@login.explorer.northeastern.edu
```

   Replace `c2193` with your actual GPU node name.

5. Access the application:

   Open your web browser and navigate to `http://localhost:8501` (make sure to use `http` and not `https`). You should see the Streamlit chatbot interface:



6. Verify GPU usage: To see whether the chatbot is leveraging the GPU, open another terminal window, SSH into the GPU node, and attach to your running job:

```
srun --jobid=<your_job_id> --overlap --pty /bin/bash
```

   Replace `<your_job_id>` with the job ID printed when you ran `srun` in Section 1, Step 2. Then run:

```
watch -n 1 nvidia-smi
```

   When you submit a query in the chatbot, you should see GPU utilization increase when you've issued a query to the LLM (the LLM processes it on the GPU). It is not a prerequisite for this lab for everything to run on the GPU, but you will definitely want to do so in your final project, as latency is counted in the final grade.

5 Troubleshooting

1. `ollama pull` fails silently / models don't load

   This is caused by incorrect proxy configuration inside the container. The Explorer Squid proxy must be passed in for `ollama pull` to reach `ollama.com`, while simultaneously being bypassed for localhost so Ollama's internal communication is not intercepted. Symptoms include the container starting but the model not responding, or `ollama pull` appearing to succeed but no model being available.

   Fix: Ensure you are using the full `apptainer run` command from Section 4, Step 3, which passes `HTTP_PROXY`, `HTTPS_PROXY`, and `NO_PROXY` into the container (see Section 3).

2. Port 11434 Already in Use

   If you see the error:

```
Error: listen tcp 0.0.0.0:11434: bind: address already in use
```

   This means another Ollama process is running. This can happen if:

   - Your container didn't shut down cleanly
   - You tested Ollama outside the container
   - Another student on the same GPU node is using that port

   Solution 1: Kill orphaned processes (if it's your own)

```
# Kill any running Ollama processes
pkill ollama

# Verify port is free
lsof -i :11434

# Run container again
apptainer run --nv \
  --bind "/scratch/$USER:/scratch/$USER" \
  --env OLLAMA_MODELS=/scratch/$USER/ollama_models \
  --env HTTP_PROXY=$HTTP_PROXY \
  --env HTTPS_PROXY=$HTTPS_PROXY \
  --env NO_PROXY=localhost,127.0.0.1,0.0.0.0 \
  serving-llama.sif
```

   Solution 2: Use a different port (if another student is using it)

   Multiple students may be allocated to the same GPU node. If another student is actively using port 11434, you can modify `entrypoint.sh` to use a different port:

```
# Edit entrypoint.sh and change the Ollama host port
export OLLAMA_HOST=0.0.0.0:11435  # Or 11436, 11437, etc.
```

   Then rebuild and run:

```
apptainer build serving-llama.sif serving-llama.def
apptainer run --nv \
  --bind "/scratch/$USER:/scratch/$USER" \
  --env OLLAMA_MODELS=/scratch/$USER/ollama_models \
  --env HTTP_PROXY=$HTTP_PROXY \
  --env HTTPS_PROXY=$HTTPS_PROXY \
  --env NO_PROXY=localhost,127.0.0.1,0.0.0.0 \
  serving-llama.sif
```

Note: If you change the port, you'll also need to update your SSH tunnel command to match the new port.

3. Models disappeared between sessions

   `/scratch` is purged on the first Tuesday of every month. Re-run the container. The model will re-download on first startup (the proxy flags in the `apptainer run` command ensure this succeeds).

6 Submit Your Lab

Let us know which LLM that you used and upload app.py with a screenshot of the LLM in action to Gradescope:

7 Additional Notes

- HPC gives you free GPU access – No GCP credits required
- No VM management – No need to start/stop instances
- No public web access – Requires SSH tunneling (only you can access)
- Session time limits – Maximum 8 hours on courses-gpu partition
- For development/testing only – Not suitable for production deployment
- For your final project, you can develop on HPC to save credits, and deploy on GCP for public demos

8 Getting Help

1. HPC Technical Issues: rchelp@northeastern.edu
2. HPC Documentation: `https://rc-docs.northeastern.edu`
3. Lab Questions: Contact the professor or a TA